

《操作系统A》机考函数调用手册

1 系统调用

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

ssize_t bytesread = read(0, buffer, BUFFER_SIZE);
write(1, msg, strlen(msg) + 1);

fd = fopen("123.txt", "w");
fprintf(fd, "text: %s\n", msg);
```

2 进程调用

```
#include <stdio.h>      // perror()
#include <stdlib.h>     // exit()
#include <unistd.h>     // fork(), execl(), execlp(), execv(),
execvp(), getpid(), getppid()
#include <sys/types.h>  // pid_t (也被其他头包含, 但推荐写上)
#include <sys/wait.h>   // wait(), waitpid()
#include <signal.h>     // kill(), SIGKILL

pid_t pid;
pid = fork();

execl("./child", "child", "a para");
execlp("ls", "ls", "a para");
char *args[] = {"child", "a para"};
execv("./child", args);
char *args[] = {"ls", "a para"};
execvp("ls", args);

exit(0);

pid = getpid();
pid = getppid();
pid = wait(NULL);
pid = waitpid(pid, NULL, 0);
kill(pid, SIGKILL);
perror("failed!");
```

3 线程调用

```
#include <pthread.h>
#include <sched.h>
#include <unistd.h>    // 提供 sleep()
#include <stdlib.h>    // 可选, 用于 malloc/free 或 exit()
#include <stdio.h>    // 可选, 用于 printf/debug

pthread_t tid;
pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_create(&tid, &attr, function, (void *)args);
pthread_yield();
sleep(num);
pthread_exit((void *)status);
pthread_join(tid, (void **)status);
```

4 同步与互斥

4.1 pthread库实现mutex锁

```
#include <pthread.h>

pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);
pthread_mutex_lock(&mutex);
pthread_mutex_unlock(&mutex);
pthread_mutex_destroy(&mutex);
```

4.2 pthread库实现条件变量

```
#include <pthread.h>

pthread_cond_t cond;
pthread_cond_init(&cond, NULL);
pthread_cond_wait(&cond, &mutex);    // P()
pthread_cond_signal(&cond);          // V()
pthread_cond_destroy(&cond);
```

4.3 semaphore库实现信号量机制

```
#include <semaphore.h>

define N 10
sem_t empty;
sem_init(&empty, 0, N);
sem_wait(&empty);      // P()
sem_post(&empty);      // V()
sem_destroy(&empty);
```

5 进程间通信

5.1 普通管道

```
#include <unistd.h>

int pipe1[2];
pipe(pipe1);
close(pipe1[0]);
read(pipe1[0], buffer, BUFFER_SIZE);
write(pipe1[1], msg, strlen(msg) + 1);
```

5.2 有名管道

```
#include <sys/stat.h>
#include <fcntl.h>

define PATH "tmp/fifo"
mkfifo(PATH, 0666);
fd = open(PATH, O_RDONLY); // O_WRONLY
read(fd, buffer, BUDDER_SIZE);
write(fd, msg, strlen(msg) + 1);
close(fd)
unlink(PATH);
```

5.3 共享内存

```
#include <sys/ipc.h>
#include <sys/shm.h>

define PATH "./shamt"
key_t key;
key = ftok(PATH, 65);
int shmid = shmget(key, size, IPC_CREAT | 0666);
char *str = (char *)shmat(shmid, NULL, 0);
shmdt(str);
```